

Fast and Provably Accurate: High-Dimensional Euclidean Minimum Spanning Trees and HDBSCAN

Martin Aumüller
IT University of Copenhagen
Copenhagen, Denmark
maau@itu.dk

Francesco Pio Monaco
University of Trento
Trento, Italy
francescopio.monaco@unitn.it

Matteo Ceccarello*
University of Padova
Padova, Italy
matteo.ceccarello@unipd.it

Francesco Silvestri
University of Padova
Padova, Italy
francesco.silvestri@unipd.it

Abstract

The Euclidean Minimum Spanning Tree (EMST) is a fundamental problem that is widely used in tasks such as data analysis and spatial clustering. Its computation is, however, extremely expensive: in high dimensions exact algorithms incur a quadratic-time complexity, making them impractical on large datasets.

We present an *exact* algorithm for computing the EMST in high dimensions with *probabilistic* guarantees: it succeeds with a user-controlled probability $\geq 1 - \delta$. Furthermore, we present a faster *approximate* algorithm for the problem, with a controlled error on the weight of the resulting tree. Our algorithms can be used both with the standard Euclidean distance and with the *mutual reachability distance*, thus giving both exact and approximate algorithms for the HDBSCAN clustering problem. All our algorithms lend themselves to an *anytime implementation*, allowing the user to interrupt the execution when they see fit and obtain progressively better solutions the longer the computation progresses.

Experimental results on real-world datasets up to over a million data points and over 700 dimensions show that our methods outperform the state of the art and are able to exercise a controlled tradeoff between time and quality. In particular, on million-scale datasets our approximate algorithm finds a spanning tree with $\approx 5\%$ relative error in about 30 seconds.

PVLDB Reference Format:

Martin Aumüller, Matteo Ceccarello, Francesco Pio Monaco, and Francesco Silvestri. Fast and Provably Accurate: High-Dimensional Euclidean Minimum Spanning Trees and HDBSCAN. PVLDB, 14(1): XXX-XXX, 2020.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at <https://github.com/Cecca/panna>.

*Corresponding author.

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.

Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

1 Introduction

The Euclidean Minimum Spanning Tree (EMST) is a fundamental geometric primitive, where the goal is to find a minimum spanning tree of the complete weighted graph induced by the pairwise Euclidean distances between points of a D -dimensional dataset. It is a fundamental primitive for several applications, such as hierarchical clustering [9], approximating the traveling salesman problem, percolation analysis, network routing [38], discovering dark-matter halos and simulations of the early universe [28], and DNA microarray analysis [26].

The need to compute distances makes the EMST problem notoriously difficult for large, high-dimensional data. In fact, directly applying MST algorithms such as Kruskal’s or Prim’s to the complete graph is impractical for large datasets, inasmuch as it requires storing and sorting a quadratic number of edges. Furthermore, pruning edges using exact spatial indices (e.g. KD-trees) is ineffective in high dimensions due to the curse of dimensionality [24].

These efficiency limitations are felt particularly in density-based clustering, which allows us to identify dense subsets of data with arbitrary shapes. The popular single-linkage clustering algorithm is equivalent to computing the EMST [21]. The widely adopted Hierarchical Density-Based Spatial Clustering of Applications with Noise (HDBSCAN) is based on computing a minimum spanning tree where edge weights are determined by the *mutual reachability distance*, which accounts for both the Euclidean distance and the density around the two endpoints.

Given the importance of the problem, several approaches have been proposed in the literature. Shamos and Hoey [34] provide a fast exact algorithm in $O(n \log n)$ time for two-dimensional datasets, building on the Delaunay Triangulation. In higher dimensions, Chatterjee et al. [11] propose an algorithm based on the Well-Separated Pair Decomposition (WSPD). This approach has been further developed with the inclusion of parallelism by Wang et al. [39]. Another line of work builds on the Dual-Tree Borůvka algorithm [13, 28–30], which pairs Borůvka’s algorithm for the MST with indices to answer nearest neighbor queries. All of these approaches are very effective in low dimensions but suffer from degraded performance in high dimensions, due to the curse of dimensionality.

Approximation algorithms can instead be efficient in high dimensions. A first line of work focuses on approximating the total weight of the tree, without returning the actual tree [16, 23]. If one is interested in the actual tree, Vaidya [37] and subsequently

Arya and Mount [3] developed algorithms to compute a $(1 + \varepsilon)$ -approximate MST. While theoretically sound, these approaches often incur an exponential dependency on the dimension D or involve large constants that limit applicability to high dimensions. More recently, heuristic approaches using graph-based indices for approximate nearest neighbor searches [31, 32] have been proposed. Yet, a gap remains for algorithms that provide guarantees on the approximation while scaling to large, high-dimensional data.

To close this gap, we propose both exact and approximate EMST algorithms *with guarantees* based on Locality Sensitive Hashing [24]. Our exact algorithm computes the exact EMST with a user-controlled success probability $\geq 1 - \delta$. The approximate algorithm allows us to compute a spanning tree of the input points whose weight is guaranteed to be within $1 + \varepsilon$ of the minimum, with ε controlling the tradeoff between running time and accuracy.

The exact and approximate algorithms are described by the same basic approach. The algorithms first build an index based on LSH Forest [8]. Then, this index is visited in such a way that edges are likely to be discovered in order of increasing distance; such edges are used to progressively build a spanning tree of the point set. By reasoning about the probability that key edges of an optimal tree have been missed, we set up different stopping conditions on the discovery of new edges that lead to the exact and approximate variants of our algorithms. Both algorithms can be extended to support the mutual reachability distance, thus giving exact and approximate algorithms for HDBSCAN [9, 30].

Our contributions are the following:

- We develop an exact algorithm for the EMST problem that circumvents the curse of dimensionality by accepting a user-defined failure probability δ .
- We develop an approximate algorithm for the EMST problem, whose returned tree has total weight at most a factor $(1 + \varepsilon)$ larger than the optimal one. The parameter ε controls the tradeoff between accuracy and efficiency.
- We extend our algorithms to the EMST problem under the mutual reachability distance, allowing for the computation of high-dimensional HDBSCAN clustering.
- We theoretically analyze the efficiency of our algorithms in an *instance-aware* fashion, introducing a novel way of quantifying the difficulty of a *particular instance*.
- We provide a comprehensive performance assessment on high-dimensional datasets, demonstrating the scalability and effectiveness of our algorithms.

While we derive our algorithms for Euclidean distance, our framework is applicable to general metric spaces (e.g., Jaccard or angular distances) provided a *locality-sensitive* family of hash functions exists for the metric [12].

An extended version containing the omitted proofs is available alongside our source code at <https://github.com/Cecca/panna>.

2 Related Work

Exact EMST Algorithms. The baseline for naively computing the EMST of n points in a D -dimensional space requires $O(Dn^2)$ time using any MST procedure like Kruskal’s or Prim’s on a complete weighted graph. In $\mathbb{R}^2$, the problem can be solved in $O(n \log n)$ using the Delaunay Triangulation [34].

Another popular exact approach is the Dual-Tree Borůvka algorithm [28], which operates iteratively: each vertex selects its minimum weight incident edge to form connected components, repeating until only one component remains. While its $O(m \log n)$ time complexity is naturally parallelizable [13, 29], the number of edges m in a complete graph is $O(n^2)$, making it prohibitive in high dimensions without aggressive filtering.

Edge Filtering and Pruning. Several works focus on pruning unnecessary distance computations. Chatterjee et al. [11] developed GeoFilterKruskal (GFK), which improves on Kruskal’s algorithm by quickly filtering distance computations over terminals using Well-Separated Pair Decomposition (WSPD) and a Union Find data structure. More recently, Wang et al. [39] developed a parallel algorithm based on WSPD, building an efficient variant of GFK that is up to an order of magnitude faster than other implementations. These approaches depend exponentially on the data dimensionality.

Approximation Strategies. Given the *curse of dimensionality*, $1 + \varepsilon$ approximations are often preferred over exact solutions. Current methods generally approximate the total weight [23] or the tree structure itself. Vaidya [37] provides an algorithm to build a tree whose weight is a $1 + \varepsilon$ factor larger than the minimum, in time $O(n \log n + 1/\varepsilon^D \log(1/\varepsilon)n)$, thus with an exponential dependency on the dimension D of the space. Arya and Mount [3] later refined this approach to mitigate the dimensionality penalty. Czumaj et al. [16] provide a $(1 + \varepsilon)$ approximation algorithm for the *weight* of the EMST in time $\tilde{O}(\sqrt{n}/\varepsilon^{2+D/2})$, where D is the dimensionality of the Euclidean space, assuming access to additional data structures for orthogonal range queries and cone nearest neighbor queries. Har-Peled et al. [22] provided a theoretical bridge by using NN search to add edges in approximately increasing weight order. Recent trends favor graph-based indices, such as Hierarchical Navigable Small World graphs (HNSW [27]), to approximate the EMST [31, 32].

Clustering. The connection between MST and clustering is well-established, as single-linkage clustering directly derives from the MST of a graph [21]. This connection is foundational in HDBSCAN [9], a clustering algorithm that groups points that are in dense regions and close together in proximity. HDBSCAN extends the foundational Density-Based Spatial Clustering of Applications with Noise (DBSCAN) [19], transitioning from a fixed density threshold to a hierarchical approach. While the original DBSCAN algorithm identifies clusters based on a fixed proximity parameter, HDBSCAN eliminates this constraint by exploring all possible density levels simultaneously. This hierarchical approach is realized through the construction of an MST, where edge weights are based on the *mutual reachability distance*, a measure that leverages the Euclidean distance. Recent optimizations by de Berg et al. [18] achieve loglinear time complexity for $\mathbb{R}^2$ but rely on computing bichromatic closest pairs which introduce costly operations for higher dimensions. To address these scaling issues, Wang et al. [39] exploit the structural similarities between EMST construction and hierarchical clustering to adapt the parallel GFK procedure to the mutual reachability graph.

LSH Data Structures. While the aforementioned filtering methods rely on deterministic partitioning or on graph-heuristics, Locality

Sensitive Hashing (LSH) [24] offers a probabilistic alternative designed to bypass expensive all-pairs comparisons. Our work adapts the LSH forest data structure proposed by Bawa et al. [8], which dynamically adjusts hash prefix lengths to avoid rigid parameter tuning. However, a critical challenge is that LSH is optimized to discover short edges (i.e., close neighbors) and inherently filters the long edges required to link distant connected components. We overcome this challenge by implementing a progressive relaxation and cross-iteration filtering, ensuring scalability without sacrificing global connectivity. While LSH forests have seen success in vector search within other domains [5, 6, 10], our framework represents a novel extension to clustering.

3 Preliminaries

EMST and HDBSCAN. We consider a metric space $(\mathcal{X}, d)$ where $d: \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}_{\geq 0}$ is the distance function. In particular, we take $\mathcal{X} = \mathbb{R}^D$ and $d(x, y) = \|x - y\|_2$. For notational convenience, for a pair $e = (x, y)$ we let $d(e) = d(x, y)$. For $i, j \in \mathbb{N}$, we use $[i, j] := \{i, i+1, \dots, j\}$ to denote the integers from i through j .

For a given point set $S \subset \mathbb{R}^D$, consider a tree T over the graph $G = (S, S \times S)$. We define the *cost* of the tree T as

$$c(T) = \sum_{(x, y) \in T} d(x, y).$$

The Euclidean Minimum Spanning Tree problem requires identifying a tree T consisting of $n - 1$ edges where $c(T)$ is minimal. For simplicity of exposition we assume that all distances $d(x, y)$ are distinct such that the minimum spanning tree is unique.

HDBSCAN [9] is a hierarchical clustering algorithm that takes as input an additional parameter $\text{minPts} \in \mathbb{N}$. Let $d_{\text{minPts}}(x)$ be the distance to the minPts -nearest neighbor of x in S . The HDBSCAN clustering is then equivalent to computing the MST of $(S, S \times S)$ using $d_{\text{mreach}}(x, y) := \max(d_{\text{minPts}}(x), d_{\text{minPts}}(y), d(x, y))$. This distance measure is referred to as *mutual reachability distance*. Intuitively, it applies a smoothing of small distances by augmenting them with the distance required for each point to become a core point in density-based clustering [9]. In practice, we have to extract a dendrogram from the MST, which is solved by plugging the MST into existing pipelines for HDBSCAN computation (such as that of McInnes and Healy [30]).

Locality-Sensitive Hashing. An LSH family is defined as follows:

Definition 3.1 (Locality-Sensitive Hash (LSH) Family [24]). Given a distance function d , a family of hash functions $\mathcal{H} = \{h: \mathcal{X} \rightarrow R\}$ is said to be (d_1, d_2, p_1, p_2) -sensitive for $(\mathcal{X}, d)$ if for each pair of points $p, q \in \mathcal{X}$ the following statements are true:

- (1) For $d(q, p) \leq d_1$, $\Pr_{h \sim \mathcal{H}}[h(q) = h(p)] \geq p_1$.
- (2) For $d(q, p) \geq d_2$, $\Pr_{h \sim \mathcal{H}}[h(q) = h(p)] \leq p_2$.
- (3) $p_1 > p_2$ and $d_1 < d_2$.

For a given LSH family $\mathcal{H}$ and two points $x, y \in \mathcal{X}$ we let

$$p(x, y) = p(d(x, y)) = \Pr_{h \sim \mathcal{H}}[h(x) = h(y)].$$

As is standard in the analysis of LSH, we assume that we have access to the collision probability function p , and that it is a function that monotonically decreases with the distance of points.

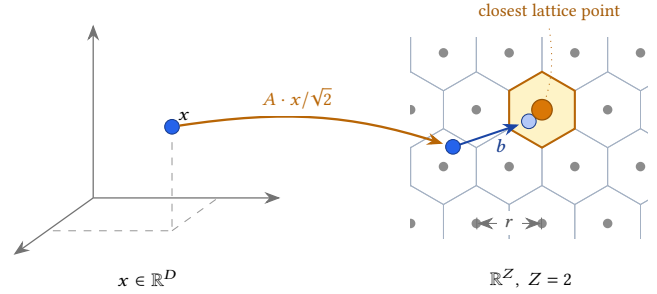


Figure 1: The hash function of Equation 1 for $Z = 2$ (hexagonal lattice). A point $x \in \mathbb{R}^D$ (left) is first projected onto $\mathbb{R}^2$ by means of $A \cdot x / \sqrt{2}$, and then translated by the random vector b . The hash value is the lattice point closest to the result, highlighted in orange.

For an integer $K \geq 1$, we consider a compound hash LSH function

$$h(x) = \langle h_1(x), h_2(x), \dots, h_K(x) \rangle.$$

Given h and an integer $i \in [0, K]$, we define the *prefix equality* operator $\stackrel{i}{=}$ as

$$h(x) \stackrel{i}{=} h(y) := \bigwedge_{t=1}^i (h_t(x) = h_t(y)).$$

In other words, $\stackrel{i}{=}$ checks whether two hash strings share the same prefix of length i . For $i = 0$ the conjunction is empty, so $h(x) \stackrel{0}{=} h(y)$ holds for every pair of points.

In the following, we assume that we are using¹ the LSH family by Andoni and Indyk [1]. Consider a Z -dimensional lattice, and a parameter $r \in \mathbb{R}_{>0}$. Let $A \in \mathbb{R}^{Z \times D}$ be a matrix with $A_{ij} \stackrel{i.i.d.}{\sim} \mathcal{N}(0, 1)$ and $b \in \mathbb{R}^Z$ s.t. $b_i \stackrel{i.i.d.}{\sim} \mathcal{U}(0, r)$. Then, the hash function is

$$\text{LATTICEDECODE} \left(\frac{A \cdot x / \sqrt{Z} + b}{r} \right) \quad (1)$$

where LATTICEDECODE returns the nearest Z -dimensional lattice point. Figure 1 illustrates the hash function for $Z = 2$, in which case the lattice of choice is the hexagonal one. For $c \geq 1$, this hash function family is (r, cr, p_1, p_2) -sensitive for Euclidean distance and its performance is characterized by $\rho := \log p_1 / \log p_2 \leq 1/c^2 + o(1)$. The table of collision probabilities can be computed offline and is independent of the dataset and its dimensionality.

4 Algorithms

We first give an algorithmic blueprint in Algorithm 1. This blueprint will then be instantiated in different ways to compute an exact Euclidean Minimum Spanning Tree (Section 4.1), an approximate EMST (Section 4.2) and an approximate EMST under the mutual reachability distance (Section 4.3).

The algorithm blueprint receives as input the dataset $S \subset \mathcal{X}$, a failure probability $\delta \in (0, 1)$ and $L \geq 1$ compound hash functions $h_1, \dots, h_L$ for some fixed $K \geq 0$, which are chosen independently

¹The widely used E2LSH by Datar et al. [17] can also be employed, at the expense of a larger number of distance computations.

Algorithm 1: EMST algorithm blueprint

Input: Dataset S ; $\delta \in (0, 1)$; hash functions $h_1, \dots, h_L$
 /* ❶ Index initialization */
 1 **for** $j = 1$ **to** L **do**
 2 $\mathcal{F}_j \leftarrow$ empty trie for repetition j ;
 3 **for** $x \in S$ **do**
 4 Compute $h_j(x)$, and store the result in $\mathcal{F}_j$, along
 with a pointer to x at the leaf;
 /* ❷ Edge discovery */
 5 $E \leftarrow \emptyset$;
 6 **for** $i = K$ **down to** 0 **do**
 7 **for** $j = 1$ **to** L **do**
 8 $E \leftarrow \text{UpdateGraph}(\mathcal{F}_{j,i}, E)$;
 9 **if** $G = (S, E, w)$ **is not connected** **then Continue**;
 10 $T \leftarrow$ a minimum spanning tree of $G = (S, E, w)$;
 11 **if** $\text{StoppingCondition}(T, i, j, \delta)$ **then return** T ;
 /* round $i = 0$ inspects every pair, so the
 algorithm always returns above */

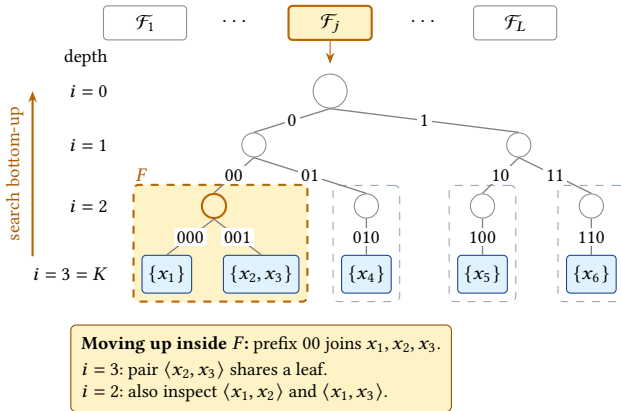


Figure 2: LSH Forest for six points using L tries and $K = 3$ binary hash functions. One trie $\mathcal{F}_j$ is expanded. Edge labels show complete hash prefixes; leaves store point indices. The four dashed boxes form $\mathcal{F}_{j,2}$. Moving upward within the highlighted subtree F reveals pairs across its leaf buckets.

at random from an LSH family $\mathcal{H}$. In this paper, for the sake of concreteness, we use the LSH family we described in Section 3, but the algorithm is agnostic to the particular LSH function used.

The algorithm proceeds in two phases. The first phase ❶ sets up an LSH Forest index data structure [8] over S , while the second phase ❷ visits this index to discover pairs of close points to use in building a spanning tree of S .

In particular, in phase ❶ for each one of the L compound hash functions received as input, we set up a trie $\mathcal{F}_j$. This trie stores, for each $x \in S$, the compound hash value $h_j(x)$: the height of the trie is thus K . At each leaf of the trie, we store the set of indices of points x sharing the same hash value. With the notation $\mathcal{F}_{j,i}$ we refer to the set of subtrees at depth i in trie $\mathcal{F}_j$. For a subtree

Algorithm 2: EMST graph update function

Data: E is the set of edges built by all previous invocations of the function, and $w_{(x,y)}$ denotes the weight of the edge between x and y .

1 **Function** $\text{UpdateGraph}(\mathcal{F}_{j,i}, E)$ **is**
 2 **for** $F \in \mathcal{F}_{j,i}$ **do** // Visit all subtrees
 3 **for** $x, y \in F : h_j(x) \stackrel{i+1}{\neq} h_j(y)$ **do**
 4 $w_{(x,y)} \leftarrow d(x, y)$;
 5 $E \leftarrow E \cup \{(x, y)\}$;
 6 **return** E ;

$F \in \mathcal{F}_{j,i}$ we denote by $x \in F$ a point $x \in S$ pointed to by a leaf of F . In particular, note that for $F \in \mathcal{F}_{j,i}$ it holds that any two points $x, y \in F$ share at least a prefix of length i of the respective hash values (i.e. $h_j(x) \stackrel{i}{=} h_j(y)$). Figure 2 gives an example instantiation of the index data structure.

The second phase ❷ builds a spanning tree over the set of points S by extracting pairs of points from the data structure constructed in the previous phase. Initially, only pairs of points residing in the same trie leaf are considered. By virtue of how each $\mathcal{F}_j$ is constructed, points in the same leaf are likely to be at a smaller distance than points in different leaves. In subsequent iterations, the algorithm considers, in a bottom-up fashion, pairs of points in the same subtrees; by doing so, it considers points that are progressively farther away from each other. The intuition is that the algorithm thus considers pairs of points by increasing distance. This iterative process stops once the set of collected pairs of points can be used to build a spanning tree that meets a stopping condition ensuring the required guarantees are met. By virtue of this iterative process, the algorithm is *instance adaptive*, stopping earlier on easier datasets.

In more detail, the second phase starts with an empty set of edges E . It then proceeds in rounds, for $i = K$ down to 0 . In round i the subtrees at depth i are considered: for each of the L tries the algorithm picks $\mathcal{F}_{j,i}$, the set of subtrees rooted at depth i . This set is passed to a function UpdateGraph , for which we provide different implementations to customize the behavior of the algorithm. This function updates E with newly discovered pairs of points: different strategies for setting the weights of the new edges will be used by different algorithms. If the weighted graph built with the edges E is connected, the algorithm builds a minimum spanning tree T on it (we shall later see how this can be implemented efficiently). This minimum spanning tree is then passed to a function StoppingCondition , along with the iteration indices i and j and the failure probability δ . The implementation of StoppingCondition dictates whether the algorithm computes an exact or approximate EMST.

Note that in the last round, $i = 0$, all points share the empty prefix, so the first repetition of that round inspects every pair of points that has not been inspected before. At that point G is the complete graph and T is an exact minimum spanning tree of S . The algorithm always terminates without a brute-force fallback.

Algorithm 3: Exact EMST stopping condition

```

1 function StoppingCondition( $T, i, j, \delta$ ) is
2   return  $\sum_{(x,y) \in T} (1 - p(x,y)^i)^j \leq \delta$ ;

```

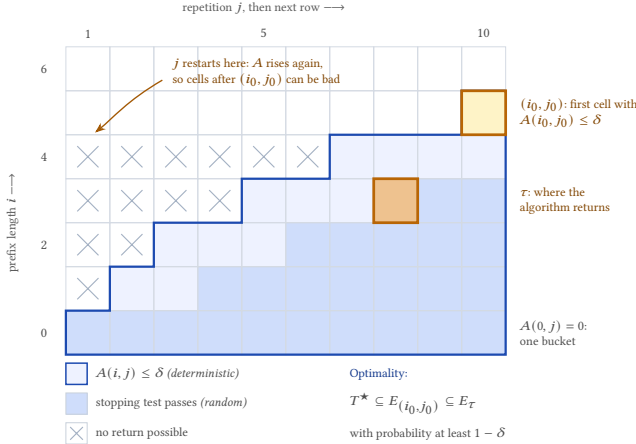


Figure 3: The checkpoint argument behind Theorem 4.1, drawn for an example instance with $K = 6$ and $L = 10$. Each cell is one iteration (i, j) of Algorithm 1, and cells are visited row by row, left to right. The quantity $A(i, j)$ decreases both with more repetitions and with shorter prefixes, so the cells with $A(i, j) \leq \delta$ form a staircase that depends only on the input and not on the sampled hash functions. Satisfying the stopping test of Algorithm 3 implies $A(i, j) \leq \delta$; the cells at which it passes are therefore contained in that staircase. Since E only grows, the event $T^* \subseteq E$ established at (i_0, j_0) still holds at τ . In the last round the prefix is empty, all points share a single bucket and $A(0, j) = 0$, so the algorithm terminates there at the latest.

4.1 Exact EMST computation

To instantiate Algorithm 1 as an exact EMST algorithm we use the UpdateGraph and StoppingCondition functions depicted in Algorithm 2 and Algorithm 3, respectively. The UpdateGraph function considers, for a set of subtrees $\mathcal{F}_{j,i}$, all pairs of points that are stored in the same subtree (and thus share a hash prefix of length i) that do not have an $i + 1$ -prefix of their hashes in common. (These pairs have been considered in the previous iteration.) For each pair under consideration we compute its distance and update the set of edges E accordingly. Note that the distance of any pair of points is computed *at most* once for a fixed trie $\mathcal{F}_j$.

The StoppingCondition function takes as input a spanning tree T , the iteration indices i and j and the failure probability δ . It then computes, for each edge of T , the probability of having missed a shorter edge in the iterations executed so far, and checks that overall this probability is smaller than the desired δ .

The following theorem ensures that the algorithm computes an exact EMST of S with probability at least $1 - \delta$. A visual representation of the core argument is given in Figure 3.

THEOREM 4.1. *Let $S \subset \mathbb{R}^D$ with $|S| = n$, and let $\delta \in (0, 1)$. Let T^* be a Euclidean Minimum Spanning Tree of S . With probability $1 - \delta$, Algorithm 1 instantiated with Algorithm 2 and Algorithm 3 as its update and stopping functions returns a tree T such that $c(T) = c(T^*)$.*

PROOF. Let $e_{(1)}^*, e_{(2)}^*, \dots, e_{(n-1)}^*$ be the edges of T^* , sorted by increasing distance. Let $e_{(1)}, e_{(2)}, \dots, e_{(n-1)}$ be the edges of T sorted by increasing distance. Since $E \subseteq S \times S$, we have that

$$d(e_{(\ell)}^*) \leq d(e_{(\ell)}) \quad \forall \ell \in [1, n-1] \quad (2)$$

otherwise the minimality of T^* would be contradicted.

For a fixed pair of indices $i \in [0, K]$ and $j \in [1, L]$, denote by $\bar{E}_{(\ell)}^*(i, j)$ the event “ $e_{(\ell)}^* \notin E$ at the end of iteration i, j ”, which happens with probability

$$\Pr[\bar{E}_{(\ell)}^*(i, j)] \leq \left(1 - p(e_{(\ell)}^*)^i\right)^j,$$

where we ignore that tries $j + 1, \dots, L$ have already been explored at prefix length $i + 1$. By the union bound, the probability of missing an edge of T^* at iteration (i, j) is bounded by

$$\Pr\left[\bigcup_{\ell=1}^{n-1} \bar{E}_{(\ell)}^*(i, j)\right] \leq \sum_{\ell=1}^{n-1} \left(1 - p(e_{(\ell)}^*)^i\right)^j \stackrel{\text{def}}{=} A(i, j).$$

For fixed parameters, $A(i, j)$ depends only on T^* and is therefore a deterministic function of the input. Whenever the algorithm returns in iteration (i, j) in Line 11, we have

$$A(i, j) = \sum_{\ell=1}^{n-1} \left(1 - p(e_{(\ell)}^*)^i\right)^j \stackrel{(a)}{\leq} \sum_{\ell=1}^{n-1} \left(1 - p(e_{(\ell)})^i\right)^j \stackrel{(b)}{\leq} \delta,$$

where (a) follows from (2) and the monotonicity of the LSH family collision probabilities, and (b) is the stopping condition of Algorithm 3. Since both steps hold for every realization of the hash functions while A is deterministic, $A(i, j) \leq \delta$ is a necessary condition for the algorithm to return in iteration (i, j) . Let (i_0, j_0) be the first iteration with $A(i_0, j_0) \leq \delta$ in the order in which Algorithm 1 enumerates iterations, which therefore is not later than (i, j) ; see Figure 3. Such an iteration exists, since $A(0, 1) = 0$. With probability at most δ we thus have $T^* \not\subseteq E$ at the end of iteration (i_0, j_0) . Taking the complement event, we have $T^* \subseteq E$ with probability at least $1 - \delta$. The algorithm cannot stop before iteration (i_0, j_0) and its edge set E only grows over the course of the algorithm. Since T is a minimum spanning tree of $G = (S, E)$ and $T^* \subseteq E$, we have $c(T) = c(T^*)$ with probability at least $1 - \delta$. $\square$

4.2 Approximate EMST computation

We now proceed to discuss our approximate EMST algorithm, which takes as additional input an approximation factor $\varepsilon \geq 0$ and guarantees to return a $(1 + \varepsilon)$ -approximation to the actual EMST with probability at least $1 - \delta$. The UpdateGraph function is the same as that used by the exact algorithm. When $\varepsilon = 0$, the same guarantee yields an *exact* EMST.

What allows us to settle for approximate answers is the function StoppingCondition depicted in Algorithm 4. For a given tree T , it partitions its edges into two sets: T_{verified} , the largest prefix (of the edges in order of increasing distance) whose edges each have

Algorithm 4: Approximate EMST stopping condition

```

1 function StoppingCondition( $T, i, j, \delta$ ) is
2   Let  $e_{(1)}, e_{(2)}, \dots, e_{(n-1)}$  be the edges of  $T$  in increasing
   order of distance;
3    $\ell \leftarrow \max \left( \{0\} \cup \{z \in [1, n-1] : (1 - p(e_{(z)}))^i \leq \frac{\delta}{n-1}\} \right)$ ;
4   if  $\ell = 0$  then return false;
5    $T_{\text{verified}} \leftarrow \{e_{(1)}, e_{(2)}, \dots, e_{(\ell)}\}$ ;
6    $T_{\text{arbitrary}} \leftarrow \{e_{(\ell+1)}, \dots, e_{(n-1)}\}$ ;
7   return  $c(T) \leq (1 + \varepsilon) (c(T_{\text{verified}}) + |T_{\text{arbitrary}}| \cdot d(e_{(\ell)}))$ ;

```

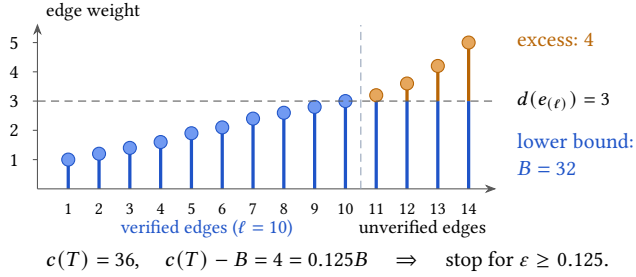


Figure 4: The rank-based stopping certificate in Algorithm 4. The blue stems represent a lower bound B on the optimal tree weight. The algorithm stops when the orange excess is at most εB , even if some edges remain unverified.

miss probability bound at most $\delta/(n-1)$; and $T_{\text{arbitrary}}$, for which no guarantees can be given. Then the function checks whether the contribution of the $T_{\text{arbitrary}}$ edges is small enough by considering this hypothetical scenario: “if all the $T_{\text{arbitrary}}$ edges were replaced by an edge as heavy as the heaviest in T_{verified} , what would be the cost of the tree?” The algorithm stops if $c(T)$ is at most $(1 + \varepsilon)$ times this hypothetical cost, see Figure 4.

The following theorem establishes the approximation guarantee.

THEOREM 4.2. *Let $S \subset \mathbb{R}^D$ with $|S| = n \geq 2$, and let $\delta \in (0, 1)$ and $\varepsilon \geq 0$. Let T^* be a Euclidean Minimum Spanning Tree of S . Algorithm 1 instantiated with Algorithm 2 and Algorithm 4 as its update and stopping functions always terminates and, with probability at least $1 - \delta$, returns a tree T such that $c(T) \leq (1 + \varepsilon)c(T^*)$.*

PROOF. Fix T^* and order its edges $e_{(1)}^*, \dots, e_{(n-1)}^*$ by increasing distance. For each $z \in [1, n-1]$, let (i_z, j_z) be the first iteration, in the order of Algorithm 1, such that

$$\left(1 - p\left(e_{(z)}^*\right)\right)^{i_z} \leq \frac{\delta}{n-1}.$$

These checkpoints exist because the failure bound is zero at $(0, 1)$. Re-using the events defined in the proof of Theorem 4.1, a union bound gives

$$\Pr \left[\bigcup_{z=1}^{n-1} \bar{E}_{(z)}^*(i_z, j_z) \right] \leq \sum_{z=1}^{n-1} \frac{\delta}{n-1} = \delta.$$

Thus, with probability at least $1 - \delta$, every reference edge has been discovered by its respective checkpoint.

Algorithm 5: Graph update function with mutual reachability distance

```

1 Function UpdateGraph( $\mathcal{F}_{j,i}, E$ ) is
2    $H \leftarrow E \cup \{\langle x, y \rangle : y \in \mathcal{N}_x, x \in S\}$ ;
3   for  $F \in \mathcal{F}_{j,i}$  do // Visit all subtries
4     for  $x, y \in F : h_j(x) \neq h_j(y)$  do
5        $a \leftarrow d(x, y); H \leftarrow H \cup \{\langle x, y \rangle\}$ ;
6        $\mathcal{N}_x.\text{insert}(y, a); \mathcal{N}_y.\text{insert}(x, a)$ ;
7   for  $\langle x, y \rangle \in H$  do
8      $w_{(x,y)} \leftarrow \max\{d(x, y), \mathcal{N}_x.\text{max}(), \mathcal{N}_y.\text{max}()\}$ ;
9   return a minimum spanning tree of  $G(S, H, w)$ ;

```

Consider now the iteration (i, j) in which the algorithm returns T , and let $e_{(1)}, \dots, e_{(n-1)}$ be its edges sorted by increasing distance. Let $\ell \geq 1$ be the rank selected by Algorithm 4. For every $z \leq \ell$, optimality of T^* and monotonicity of the collision probabilities give

$$\left(1 - p\left(e_{(z)}^*\right)\right)^i \leq \left(1 - p\left(e_{(z)}\right)\right)^i \leq \frac{\delta}{n-1}.$$

Thus (i_z, j_z) occurs no later than (i, j) . On the event established above, all reference edges $e_{(1)}^*, \dots, e_{(\ell)}^*$ belong to the current graph. The consequence is that

$$c(T_{\text{verified}}) = \sum_{z=1}^{\ell} d\left(e_{(z)}^*\right) \quad (3)$$

and that $d(e_{(\ell)}) = d(e_{(\ell)}^*)$, hence

$$d(e_{(\ell)}) \leq d(e_{(z)}^*) \quad \forall z \in [\ell+1, n-1]. \quad (4)$$

Therefore, we have:

$$\begin{aligned}
c(T) &\stackrel{(a)}{\leq} (1 + \varepsilon) (c(T_{\text{verified}}) + |T_{\text{arbitrary}}| \cdot d(e_{(\ell)})) \\
&\stackrel{(b)}{\leq} (1 + \varepsilon) \left(c(T_{\text{verified}}) + \sum_{z=\ell+1}^{n-1} d(e_{(z)}^*) \right) \\
&\stackrel{(c)}{=} (1 + \varepsilon) c(T^*)
\end{aligned}$$

where (a) is the stopping condition, (b) follows from (4) and (c) follows from (3). Figure 4 visualizes this argument. $\square$

4.3 Approximate EMST under the mutual reachability distance

For HDBSCAN [9] we need to build an EMST under the mutual reachability distance. The gist of our algorithm is that it computes progressively refined approximations of both the spanning tree and the core distances at the same time.

To this end, we instantiate the blueprint algorithm with the graph update subroutine depicted in Algorithm 5. This algorithm maintains for each point x a priority queue $\mathcal{N}_x$ of the minPts closest distinct inspected neighbors, with consistent tie-breaking. For initialization, choose minPts distinct other points for each point and collect the resulting pairs in E_0 , adding arbitrary edges to make (S, E_0) connected. Insert every pair of E_0 into both endpoint queues,

and initialize E as a minimum spanning tree of (S, E_0) under the resulting mutual-reachability weight estimates. The maximum of each priority queue $N_x.\text{max}()$ is an upper bound on the true core distance which can only decrease as the algorithm progresses.

When updating the graph, Algorithm 5 first creates a set H of candidate edges (Line 2), including the edges of the current candidate spanning tree E and all the $n \cdot \text{minPts}$ edges in the core-distance priority queues, taken as a snapshot before any insertions. For each edge visited in a repetition (Line 4) the priority queues of each endpoint are updated, and the edge is added to the candidate set H . Then, the weights of all edges in H are computed in Line 8 using the final core distance estimates of this repetition, providing an upper bound on the mutual reachability distance of each pair. The edges of the updated graph are then obtained by finding a minimum spanning tree of the graph induced by H (Line 9). This bounds the number of retained edges.

Then, this spanning tree is passed to Algorithm 4, with failure probability budget $\delta/(2\text{minPts} + 1)$, to test for termination using the current weight estimates w .

The following theorem ensures that this algorithm finds a $(1 + \varepsilon)$ -approximate spanning tree under the mutual reachability distance.

THEOREM 4.3. *Let $S \subset \mathbb{R}^D$ with $|S| = n$, and let $\delta \in (0, 1)$, $\varepsilon \geq 0$, and $0 < \text{minPts} \leq n - 1$. Let T^* be a minimum spanning tree of S under the mutual reachability distance. Algorithm 1 instantiated with Algorithm 5 and Algorithm 4, invoked with failure probability $\delta/(2\text{minPts} + 1)$, as its update and stopping functions always terminates and, with probability at least $1 - \delta$, returns a tree T such that $c(T) \leq (1 + \varepsilon)c(T^*)$ under the mutual reachability distance.*

PROOF. First, we need to establish a *retention invariant* showing that discarding edges preserves the minimum spanning-tree weight under the current weights. Let $\widehat{E}$ contain all inspected pairs, including E_0 . Inductively, E is a minimum spanning forest of $(S, \widehat{E})$ under current weights. Indeed, any old edge $\langle x, y \rangle$ omitted from the candidates is outside both old queues, so $d(x, y) \geq \max\{N_x.\text{max}(), N_y.\text{max}()\}$: its weight is already fixed. Its endpoints are joined in the old forest by a path of no greater weight, whose edges can only become cheaper. Thus every omitted edge has such a path among the candidates, preserving connectivity at every weight threshold. Kruskal therefore preserves the invariant, which holds initially by construction.

We are now ready to prove the theorem. Let $e_{(1)}^*, \dots, e_{(n-1)}^*$ be the edges of T^* sorted by increasing mutual reachability distance and let $a_z = d_{\text{mreach}}(e_{(z)}^*)$.

For $e_{(z)}^* = \langle x, y \rangle$, let W_z consist of $\langle x, y \rangle$ together with the pairs joining x , respectively y , to their minPts nearest neighbors, so that $|W_z| \leq 2\text{minPts} + 1$. Every pair of W_z is at Euclidean distance at most a_z , since the distance from x to any of its minPts nearest neighbors is at most the core distance of x , which is at most a_z ; the same holds for y . Call $e_{(z)}^*$ *settled* at iteration (i, j) if all pairs of W_z have been inspected by the end of that iteration. If $e_{(z)}^*$ is settled, then $e_{(z)}^* \in \widehat{E}$ and the queues N_x and N_y report exact core distances, so its weight in the conceptual graph is $w_{(x,y)} = a_z$. It need not be physically retained: the retention invariant ensures that the computed forest is a minimum spanning forest of this conceptual graph.

Let $\alpha = \delta/((2\text{minPts} + 1)(n - 1))$ be the per-rank threshold of Algorithm 4 when invoked with failure probability $\delta/(2\text{minPts} + 1)$. For each z , let (i_z, j_z) be the first iteration, in the order of Algorithm 1, with $(1 - p(a_z)^{i_z})^{j_z} \leq \alpha$. These iterations are deterministic. By a union bound over W_z , the probability that $e_{(z)}^*$ is not settled at (i_z, j_z) is at most $(2\text{minPts} + 1)\alpha = \delta/(n - 1)$. A union bound over the $n - 1$ reference edges shows that, with probability at least $1 - \delta$, every reference edge is settled by its respective checkpoint. Settlement persists: the conceptual set $\widehat{E}$ only grows and exact core estimates remain exact after further queue updates. At prefix length zero all pairs are inspected, so all reference edges are settled and the stopping condition passes with $\ell = n - 1$.

Consider the iteration (i, j) in which the algorithm returns T , and let $e_{(1)}, \dots, e_{(n-1)}$ be its edges sorted by increasing current weight w . Let $\ell \geq 1$ be the rank selected by Algorithm 4. Since the stored weights w never underestimate true mutual reachability distances and the retained tree is an MST of $(S, \widehat{E})$, sorted MST domination gives $a_z \leq w(e_{(z)})$. For every $z \leq \ell$, the uniform threshold and monotonicity of p imply

$$(1 - p(a_z)^i)^j \leq (1 - p(w(e_{(z)}))^i)^j \leq \alpha.$$

Thus (i_z, j_z) occurs no later than (i, j) . On this event, the reference edges $e_{(1)}^*, \dots, e_{(\ell)}^*$ are therefore settled: they belong to $\widehat{E}$, and w agrees with the true mutual reachability distance on them while dominating it on every other pair. This is all that the argument in the proof of Theorem 4.2 uses, and it gives $c_w(T) \leq (1 + \varepsilon)c(T^*)$, where c_w denotes the cost under w . Since w never underestimates the mutual reachability distance, the cost of T under the mutual reachability distance is at most $c_w(T)$, which concludes the proof. $\square$

5 Performance analysis

We now analyze the running time of the proposed algorithms. We analyze the construction costs and then provide a model to analyze the running time of the MST retrieval step. First, we will analyze the exact MST algorithm, and then treat approximate EMST and the mutual-reachability MST used by HDBSCAN separately.

5.1 Running Time

Construction. To implement our algorithms, we make use of the LSH Forest data structure [8]. Given the dataset S of size n and L hash functions of compound length K , we build L tries. Trie j is built on the n strings

$$\{(h_{j,1}(x), h_{j,2}(x), \dots, h_{j,K}(x)) \mid x \in S\}$$

and requires evaluating nK hash functions. At each node in the trie we use two integers to store the endpoints of the contiguous range of points that share the same hash prefix. We assume that following a child/parent pointer in the tree takes time $O(1)$. The sorted array (by hash code) of n pointers to the dataset is explicitly stored with the trie. In this way, we can access all points colliding on a prefix in constant time per point.

Spanning Tree Identification. Given the data structure described above, we implement Algorithm 1 as follows. We maintain the edge set E used by the algorithms as a dictionary with (amortized) constant-time insert and lookup time. In each iteration (i, j) , the

algorithm identifies the set $E_{i,j}$, i.e., the pairs of colliding keys under the length- i prefix of h_j that did not collide under the length- $(i+1)$ prefix. We can retrieve these pairs in time $O(|E_{i,j}|)$ by following a parent pointer from depth $i+1$ to depth i in the trie and by then enumerating all pairs of points in the contiguous array segments of siblings. Define $C = \sum_{(i,j)} |E_{i,j}|$ as the number of colliding pairs retrieved over all iterations (i, j) carried out by the algorithm. Since E is implemented as a dictionary with amortized constant insert time and lookup time, maintaining E takes time $O(C)$, and each retrieved pair requires at most one distance computation, so at most C distance computations are carried out.

We maintain a minimum spanning forest T of the evolving graph $G = (S, E)$ by using a Link/Cut tree [35]. This data structure solves connectivity and max edge queries in $O(\log n)$ time. For each edge $e \in E_{i,j}$, we add e to T if it connects disconnected components. If $e = (x, y)$ is inside a connected component of (S, T) , we retrieve the maximum edge of the path between x and y and compare it to the weight of (x, y) . If (x, y) is lighter, we unlink the maximum edge on the path and link (x, y) in the tree. As soon as $|T| = n-1$, the graph (S, T) is connected and T represents the minimum spanning tree of the current edge set E . From that point on, the termination criteria will be checked. In total, maintaining the data structure over all observed edges takes time $O(C \log n)$. Since T has at most $n-1$ edges, checking the termination criterion takes time $O(n)$ (since the edges of T are already sorted) in each of the $(K+1) \cdot L$ possible iterations. This also allows us to compute the gap information of the approximate stopping criteria.

For the approximate EMST under mutual reachability distance, an iteration inspecting c new pairs rebuilds an MST on at most $n-1 + n \cdot \text{minPts} + c$ candidate edges. Queue updates, weight updates, and Kruskal's algorithm take $O((n \cdot \text{minPts} + c) \log n)$ time per iteration, over at most $(K+1) \cdot L$ iterations.

Running time. The following theorem summarizes these bounds.

THEOREM 5.1. *Let $K, L \geq 1$ be two integers and let $S \subseteq X$ with $|S| = n$. The LSH Forest data structure can be built using $O(nKL)$ hash evaluations and $O(nKL)$ additional time. The running time of the exact and approximate EMST algorithms is $O(nKL + C \log n)$, where C is the number of colliding pairs inspected by each algorithm. The running time for the approximate EMST algorithm under mutual reachability distance is $O((nKL \cdot \text{minPts} + C) \log n)$.*

We remark here that the running time bounds are split up into a data-independent part $O(nKL)$ which is spent for building the data structure and for additional book-keeping tasks when identifying the approximate MST. The data-dependent part lies in the number of retrieved edges and distance computations to compute edge weights, and the termination criterion. In the following section, we propose models to analyze this component of the running time.

5.2 Distance computations analysis

Characterizing the data distribution. We parameterize the complexity of the algorithms in terms of two key quantities that describe the distribution of distances between the points of a point set S . This allows us to characterize the performance of the algorithms beyond their worst-case behavior [33].

First, for a point set S , we denote by $e_{(z)} \in S \times S$ the pair of points at rank z in the list of pairs of points of S sorted by increasing

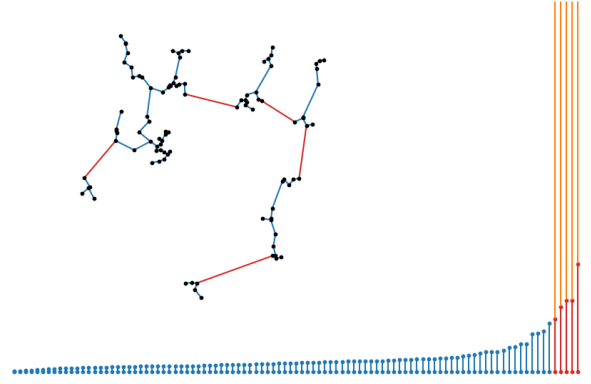


Figure 5: Example of ε -flexibility for a synthetic 2-dimensional dataset of 100 points for $\varepsilon = 2$. The edges of the EMST are shown both connecting their endpoints, and arranged by increasing weight. The red edges are the ones that can be replaced by arbitrary ones (in orange their worst-case weight, i.e. the diameter) while being at most a fraction $\varepsilon = 2$ of the total cost of the blue edges.

distance. Therefore $e_{(1)}$ is the closest pair of points, and $e_{(\binom{n}{2})}$ is the farthest. For an integer ℓ we denote by $E_{(\leq \ell)}$ the set of pairs that have a rank less than or equal to ℓ in the aforementioned ranking. Write $t_{(1)}, t_{(2)}, \dots, t_{(n-1)}$ for the edges of T sorted by weight.

We begin by introducing a novel measure to capture the difficulty of approximating a minimum spanning tree.

Definition 5.2. For a set of points S , let $\Phi(S)$ be its diameter and let T be a spanning tree over S . Given $\varepsilon \geq 0$, define the ε -confirmed length $\ell_\varepsilon(T)$ as the smallest ℓ such that $(n-1-\ell)\Phi(S) \leq \varepsilon \sum_{i=1}^{\ell} d(t_{(i)})$. The ε -flexibility of T is $\psi_\varepsilon(T) := n-1-\ell_\varepsilon(T)$.

In other words, the ε -flexibility is the largest integer such that if we replaced the heaviest $\psi_\varepsilon(T)$ edges of the tree with arbitrary ones, in the worst case the total cost of the replacements would be at most a fraction ε of the cost of the rest of the tree. Figure 5 shows an example. For a tree with a high ε -flexibility it is easy to get a $(1+\varepsilon)$ -approximation, since many of the optimal edges can be replaced by arbitrary ones. Clearly, for $\varepsilon = 0$ we have $\psi_\varepsilon(T) = 0$. While ε -flexibility dictates which edges in T are strictly necessary, the algorithmic cost of finding them depends on the distribution of distances in the data up to the maximum required weight, $d_\varepsilon = d(t_{(\ell_\varepsilon(T))})$. We characterize this distribution using *edge mass*.

Definition 5.3. Let S be a set of points, and let $r > 0$ be a distance threshold. The *edge mass at r* is defined as

$$\mu_r = \left| \left\{ e \in \binom{S}{2} : d(e) \leq r \right\} \right|.$$

In other words, for a distance threshold r the edge mass μ_r is the number of point pairs whose distance does not exceed r . By evaluating the edge mass specifically at $r = d_\varepsilon$ (denoted μ_{d_ε}), we can parametrize the complexity of our approximate algorithm.

Definition 5.4. For a dataset S and an integer $\ell \in [1, \binom{n}{2}]$, let $\ell' = \min \{2\ell, \binom{n}{2}\}$. Let $e_{(1)}, e_{(2)}, \dots, e_{(\ell')}$ be the sequence of pairs of points of S ordered by increasing distance. The *contrast* at ℓ is

$$\text{contrast}_\ell(S) = \frac{d(e_{(\ell')})}{d(e_{(\ell)})}.$$

Analyzing the exact algorithm. We bound the work of the exact algorithm by a full sweep through a hypothetical, fixed checkpoint that assumes that input distances are all known. To this end, fix an EMST T^* and let $d_{\max}$ be its largest edge weight. Set $\mu = \mu_{d_{\max}}$, $c = \text{contrast}_\mu(S)$, $p_1 = p(d_{\max})$, and $p_2 = p(c d_{\max})$, and let $\rho = \log(1/p_1)/\log(1/p_2)$. Define the *oracle parameters*

$$K^* = \left\lceil \frac{\log(n^2/\mu)}{\log(1/p_2)} \right\rceil, \quad L^* = \left\lceil \frac{\log(n/\delta)}{p_1^{K^*}} \right\rceil. \quad (5)$$

The conceptual procedure $\text{ORACLE}(S, d_{\max}, \mu, \delta)$ inspects collisions in L^* independent tries at depth K^* and computes an MST using the retrieved edges of length at most $d_{\max}$. Each edge of T^* is missed with probability at most $(1 - p_1^{K^*})^{L^*} \leq \delta/n$; hence the procedure recovers T^* with probability at least $1 - \delta$. We use its checkpoint (K^*, L^*) directly to analyze the exact algorithm. The intuition of the proof is that Algorithm 1 with the Euclidean distance update and the exact stopping rule does a limited amount of work more than the hypothetical ORACLE procedure outlined above.

THEOREM 5.5. Let $|S| = n \geq 2$, $\delta \in (0, 1)$, and define μ, p_1, c as above. Assume $0 < p_2 \leq p_1 < 1$ and $p_1 = \Omega(1)$. Run Algorithm 1 with the Euclidean distance update rule and exact stopping condition, and assume that $K \geq K^*$ and $L \geq L^*$ from (5). With probability $\geq 1 - \delta$, the algorithm returns an exact EMST no later than (K^*, L^*) , in which case the algorithm carries out at most

$$O\left((n + \mu)^{1+1/c^2+o(1)} \log \frac{n}{\delta} + L\mu\right)$$

distance computations in expectation.

PROOF. Fix T^* and reuse $A(i, j) = \sum_{e \in T^*} (1 - p(e)^i)^j$ from the proof of Theorem 4.1. As before, let (i_0, j_0) be the first iteration in the order in which the algorithm enumerates the iterations such that $A(i_0, j_0) \leq \delta$. With probability at least $1 - \delta$, T^* is part of the edge set at (i_0, j_0) . Assume that this is the case. Note that at $h = (K^*, L^*)$, the choice of parameters gives $A(h) \leq (n - 1)e^{-L^* p_1^{K^*}} \leq \delta$. So, (i_0, j_0) may only precede h and the algorithm is guaranteed to stop no later than at h .

Let C count colliding pairs retrieved through h in the full sweep, assuming that we are continuing the algorithm beyond (i_0, j_0) . A pair is retrieved at most once per trie over all depths. At h , the first L^* tries have been explored through depth K^* , and the remaining $(L - L^*)$ ones through depth $K^* + 1$. Consequently,

$$\mathbb{E}[C] \leq L^* \sum_{e \in S \times S} p(e)^{K^*} + (L - L^*) \sum_{e \in S \times S} p(e)^{K^*+1}. \quad (6)$$

For $K = K^*$ the unvisited tries contribute nothing, so the second term is absent and the same bound holds. The first term is the oracle's collision work; the second covers the extra tries.

Let us focus on the first term of the sum. If $2\mu < \binom{n}{2}$, the first 2μ pairs contribute at most 2μ . By the definition of contrast, every remaining pair has distance at least $c d_{\max}$, so its collision probability

is at most $p_2^{K^*}$. For the parameter setting in (5), we get $p_2^{K^*} \leq \mu/n^2$, hence the sum is bounded by at most 3μ . If $2\mu \geq \binom{n}{2}$, bounding each summand by one gives the same conclusion. In particular, this part of the sum contributes at most $3\mu L^*$. It remains to express the oracle term in the stated parameters. Rounding K^* up increases $p_1^{-K^*}$ by at most a factor $1/p_1$:

$$p_1^{-K^*} \leq \frac{1}{p_1} \left(\frac{n^2}{\mu}\right)^\rho, \quad L^* \leq 1 + \frac{1}{p_1} \left(\frac{n^2}{\mu}\right)^\rho \log \frac{n}{\delta}.$$

Since $0 \leq \rho \leq 1$, $n^{2\rho} \mu^{1-\rho} \leq (n + \mu)^{1+\rho}$. Using $p_1 = \Omega(1)$ and the LSH bound $\rho \leq 1/c^2 + o(1)$ from Section 3, and bounding $(L - L^*) \sum_{e \in S \times S} p(e)^{K^*+1} \leq (L - L^*) \sum_{e \in S \times S} p(e)^{K^*} \leq 3\mu(L - L^*) \leq 3\mu L$, gives

$$\mathbb{E}[C] = O\left((n + \mu)^{1+1/c^2+o(1)} \log \frac{n}{\delta} + L\mu\right),$$

as claimed. $\square$

The theorem shows that the exact EMST algorithm achieves truly subquadratic running time for $\mu = n^\alpha$ if $\alpha(1 + \rho) < 2$ and $L\mu = o(n^2)$. It does not lift the curse of dimensionality: For example, for $\mu = \Theta(n^2)$, it does not provide subquadratic guarantees.

Analyzing the approximate EMST algorithm. The proof idea for the exact algorithm carries over to the approximate variants as well. In particular, given full knowledge of the data distribution, we can find a single good iteration $(K_\varepsilon^*, L_\varepsilon^*)$ on which the oracle algorithm guarantees that it has found the desired approximate results with probability $1 - \delta$. Next, we need to prove that our adaptive variants do not end later than this (conditioned on the successful event), and then analyze their running time as if they were run all the way up to iteration $(K_\varepsilon^*, L_\varepsilon^*)$.

For the approximate variant, let $\ell_\varepsilon(T^*)$ be the ε -confirmed length of the EMST T^* of S . Let d_ε be the distance of the edge at weight rank $\ell_\varepsilon(T^*)$, let μ_{d_ε} be the corresponding edge mass, and consider the contrast $c_\varepsilon = \text{contrast}_{\mu_{d_\varepsilon}}(S)$. Using these values in (5), referring to them as K_ε^* and L_ε^* , guarantees that at $(K_\varepsilon^*, L_\varepsilon^*)$, the first $\ell_\varepsilon(T^*)$ edges are correctly identified, and the running time analysis carries over *mutatis mutandis*.

COROLLARY 5.6. Let $|S| = n \geq 2$, $\delta \in (0, 1)$, and let $\varepsilon \geq 0$ be constant. Run Algorithm 1 with $K \geq K_\varepsilon^*$, $L \geq L_\varepsilon^*$, the Euclidean distance update rule, and Algorithm 4 with budget δ . With probability at least $1 - \delta$, it returns a $(1 + \varepsilon)$ -approximate EMST no later than $(K_\varepsilon^*, L_\varepsilon^*)$, using

$$O\left((n + \mu_{d_\varepsilon})^{1+1/c_\varepsilon^2+o(1)} \log \frac{n}{\delta} + L\mu_{d_\varepsilon}\right)$$

distance computations in expectation.

Analysis under mutual reachability distance. The analysis under mutual reachability distance is slightly more involved since it uses not only the Euclidean distance but also the minPts -nearest neighbor distances. Fix a mutual-reachability MST T^* with sorted weights $a_1, \dots, a_{n-1}$ according to mutual reachability distance. Let $\ell = \ell_\varepsilon(T^*)$, using these weights, and set $R = a_\ell$. Let μ_R count the number of pairs at Euclidean distance at most R , and set the contrast c_ε in relation to μ_R .

Given failure probability $\delta \in (0, 1)$, recall from Theorem 4.3 that we run the stopping criterion check with a budget $\delta' = \delta/(2\text{minPts} +$

1). Now, set the oracle parameters in (5) using distance R and μ_R , with δ' as budget.

COROLLARY 5.7. *Let $|S| = n \geq 2$, $\text{minPts} \in \{1, \dots, n-1\}$, $\delta \in (0, 1)$, and let $\epsilon \geq 0$ be constant. Run Algorithm 1 with $K \geq K_\epsilon^*$, $L \geq L_\epsilon^*$, the update rule of Section 4.3, and Algorithm 4 with budget $\delta/(2\text{minPts} + 1)$. With probability at least $1 - \delta$, it returns a $(1 + \epsilon)$ -approximate mutual-reachability MST no later than $(K_\epsilon^*, L_\epsilon^*)$, using*

$$O\left((n + \mu_R)^{1+1/c_\epsilon^2+o(1)} \log \frac{n(2\text{minPts} + 1)}{\delta} + L\mu_R\right)$$

distance computations in expectation.

We remark that this bound can substantially overestimate the work performed in practice. Although R is a mutual-reachability weight, it is used as a Euclidean search radius: μ_R therefore also counts pairs whose mutual-reachability distance exceeds R .

6 Implementation considerations

Hash function implementation. For the LSH function family (Eq. (1)), we pick the E8 lattice, since it allows for simple constant-time decoding [14] using SIMD instructions.

For $\mathbb{R}^D$, a naive implementation of the hashing algorithm would compute the matrix-vector multiplication for all $K \cdot L$ hash functions required by the algorithm, requiring $8DKL$ multiplications (since the lattice is 8-dimensional). Let $\hat{A} \in \mathbb{R}^{8KL \times D}$ be the rectangular matrix obtained by stacking all the random $8 \times D$ matrices for the KL hash functions. Then $\hat{y} = \hat{A}x \in \mathbb{R}^{8KL}$ gives, in a single matrix-vector multiplication, all the projections required for hashing. Now, let $\tilde{D} = 2^{\lceil \log_2 \max\{D, 8KL\} \rceil}$ be the next power of two of the maximum between D and $8KL$, and let $\tilde{A} \in \mathbb{R}^{\tilde{D} \times \tilde{D}}$ be the square matrix obtained by padding $\hat{A}$ with random values $\sim \mathcal{N}(0, 1)$. Similarly, let $\tilde{x}$ be x padded with zeros to $\tilde{D}$ dimensions. Clearly, $\tilde{y} = \tilde{A}\tilde{x} \in \mathbb{R}^{\tilde{D}}$ is such that $\hat{y} = \tilde{y}_{1:8KL}$, that is the first $8KL$ components of $\tilde{y}$ contain the projections required for computing the hash values. Obviously, implementing directly this approach would be much worse than the naive one. However, by noting that $\tilde{A}$ is a random rotation of $\tilde{x}$, we can apply pseudo-random rotations as in Andoni et al. [2]. In particular, $\tilde{y} = HR_3HR_2HR_1\tilde{x}$, where R_i for $i \in \{1, 2, 3\}$ is a random diagonal ± 1 -matrix, and H is the Hadamard transform. Using the Fast Hadamard Transform, this can be computed in time $\Theta(\tilde{D} \log \tilde{D})$, faster than the naive $\Theta(DKL)$ approach².

Setting the parameters of the hash function family. The LSH family we use (Equation (1)) features a parameter r , whose setting crucially impacts performance: larger values make for a higher collision probability, making the algorithm converge faster at the expense of more distance computations, while smaller values make for a more selective LSH family (i.e. fewer distance computations) but require substantially more repetitions to bring the failure probability below δ . We experimented with several heuristics for setting r automatically, including building an EMST over a random sample of the dataset or setting r so that the average number of collisions per repetition with hash values of length K is in the order of n (since we need $n - 1$ edges to build a tree). The most effective heuristic

turned out to be the following. First, we compute a k -center clustering of the data, with $k = \sqrt{n}$ [20] (this requires time $O(n^{1.5})$). Then we build the exact EMST of the cluster centers (which requires time $O(n \log n)$, including the time to compute all-pairs-distances between the $\sqrt{n}$ centers). Finally, each node of the dataset is connected with the closest cluster center, resulting in a spanning tree of the entire dataset, whose cost is of course an upper bound to the optimal one. Let $d_{\max}$ be the heaviest edge of such a tree. We then set the parameter r so that a pair of points at distance $d_{\max}$ collides at repetition L with hashes of length 1 with probability $> 1 - \delta/n$. This ensures that the algorithms will, at latest in the last iteration (repetition L with hashes of length 1), identify as a solution this clustering-based tree. The algorithms will then likely stop earlier, since the optimal tree has shorter edges.

Running this heuristic requires ≈ 13 seconds overall on the largest dataset we use in our experiments.

Index representation. To improve data locality we store each tree as an array of node identifiers, sorted by the lexicographic order of their corresponding hash codes. After construction, hash codes are not stored explicitly: we use K bit arrays to mark the boundaries between subtrees in the array of node identifiers.

Parallelism. Within a trie, the size of subtrees is very imbalanced, hence parallelizing over the subtrees would result in severe load balancing issues [4]. Instead, for a fixed prefix length $i \in [1, K]$ (Line 7 of Algorithm 1), we run the L repetitions in batches of p , on a system with p processors. Within each batch, repetitions run in parallel, each producing candidate edges. These edges are then combined into a spanning tree by parallel tree reduction.

Duplicate candidates reduction. The same pair of points can collide in different repetitions: computing their distance is a waste of computation. Removing duplicates by means of a set data structure requires superlinear space, which is impractical for large datasets. We thus resort to maintaining the current connected components of T_{verified} (Line 5 of Algorithm 4), which requires linear space using a union-find data structure. If two colliding nodes are already connected (either directly or with a path of confirmed edges), then their distance is not computed. This reduces the number of duplicates significantly, even if it does not remove all of them.

Anytime behavior. The algorithms we present in this paper lend themselves to an *anytime* implementation, that is an implementation whose execution can be interrupted at any time, resulting in a spanning tree that is not necessarily minimum. The interesting point is that, as the computation progresses, the cost of this tree decreases monotonically. We leverage this property in designing our approximate algorithm providing formal guarantees (Section 4.2). Another possibility is to allow the user to monitor the execution, interrupting it as soon as the solution is satisfactory enough. Such a solution would still enjoy formal guarantees, with a smaller approximation factor ϵ the longer the computation goes.

7 Experiments

We aim to answer the following questions with our experimental evaluation:

RQ1 How does the solution provided by our algorithm evolve throughout its execution?

²On the setup used for our experiments, for $D = 784$, $K = 4$ and $L = 512$, the matrix multiplication approach takes $\approx 3600\mu\text{s}$ to compute all hashes for a point, while the FHT based hashing requires $\approx 90\mu\text{s}$: a 40x improvement.

- RQ2** How does our EMST algorithm compare to baselines, both in terms of running time and quality of the returned tree?
- RQ3** How does our algorithm compare with the baselines when the EMST is built using Mutual Reachability distances?
- RQ4** What is the quality of the HDBSCAN clustering built with our algorithm?

Environment and implementation. In the following we denote our algorithm *wok-stir*, for Walking Over K-prefixes for Spanning Trees by Iterative Relaxation. We perform experiments on $2 \times$ AMD EPYC 7282 (2.8 GHz) CPUs for a total of 32 cores and 512 GB of RAM. We use g++ (version 14.2) with AVX2 instructions enabled and OpenMP for parallelism. Our code is publicly available³. We set a timeout for each experiment of 24 hours.

Baselines. *MLPACK* implements the sequential Dual Tree Borůvka algorithm of March et al. [28] as part of the *mlpack* library [15]. We note that this implementation does not make use of any parallelism.

TUTTE [30], as of version 0.3.2⁴, implements both an *exact* and an *approximate* mode of execution. In the exact mode, the algorithm is based on the Dual-Tree Borůvka algorithm, where neighborhood queries are resolved using KD-trees. This makes the performance degrade severely with the increase of the dimensionality of the dataset. The *approximate* setting instead first builds a $3k$ -neighborhood graph of the dataset — where k is the parameter of the mutual reachability distance — using the *PYNNDESCENT* algorithm⁵. This approach trades exactness for much better performance on high-dimensional datasets. Output quality is not guaranteed.

HSSL [32] builds on an approximate neighborhood graph using *HNSW* [27] and computes an MST in it. Following the original paper, we define a hyperparameter $f \in \{2, 3, 4\}$ and set $M = \max\{20, f \cdot \minPts\}$ and $efC = \max\{300, 1.5 \cdot M\}$. Furthermore, we test both the plain and the self-join variants of the algorithm, in the latter case setting the number of self-join neighbors to 100. For each dataset, we report on the fastest tested configuration.

We do not include the algorithm by Wang et al. [39], which only works on low-dimensional datasets since it has an exponential dependence on the input dimension.

Datasets. The datasets we consider are *fashion* and *mnist*, both with 60 000 points and 784 dimensions, *nytimes* with 248 968 points and 256 dimensions, *sift* with 985 462 points and 128 dimensions, and *glove* with 1 183 514 points and 100 dimensions. All datasets are available from ANN-benchmarks [7].

7.1 Anytime behavior (RQ1)

As discussed in Section 6, our algorithm features an *anytime* behavior; it maintains a working solution that gets progressively refined and can prove useful at any point of the execution. To inspect this behavior more concretely, Figure 6 reports some statistics of the trees built by *wok-stir* during its execution on the *sift* dataset (others show similar behavior), under the exact setting. The y axis reports ratios to the exact EMST weight.

The total weight of the tree converges very quickly to values close to the minimum. At any point in time, the algorithm is able to

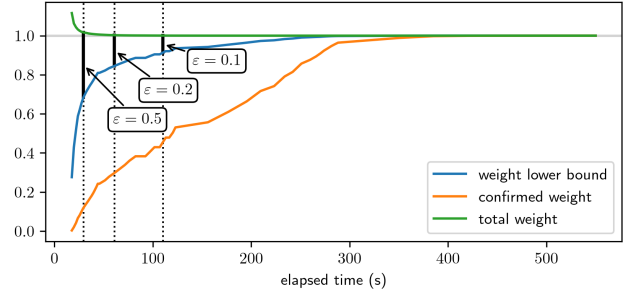


Figure 6: Evolution of the spanning tree computed by *wok-stir* on the *sift* dataset.

give guarantees on the weight of the current solution: the confirmed weight accounts for the edges that the algorithm can guarantee are settled with probability $1 - \delta$. As the figure shows, this increases as the execution progresses, as the algorithm collects more and more evidence. Furthermore, the algorithm maintains a lower bound on the weight of the optimal solution, which is used in the stopping condition (see Algorithm 4). The gap between the total weight and the lower bound measures the uncertainty of the algorithm with respect to the current solution: as more computation is invested such uncertainty decreases, eventually reaching 0 when the upper and lower bounds coincide at the end of the execution.

In principle, the algorithm could be stopped by the user at any time of the execution: the gap between the total weight of the current tree (which provides an upper bound to the optimum) and the lower bound would then give a $1 - \delta$ *confidence interval* on the weight of the returned spanning tree.

Figure 6 marks with vertical bars the stopping times of the approximate algorithm under different values of ϵ , with the black segment marking the span of the maximum relative error.

7.2 Finding the Euclidean Minimum Spanning Tree (RQ2)

We now compare *wok-stir* with the baselines for computing the Euclidean Minimum Spanning Tree. For our algorithm *wok-stir* we fix $\delta = 0.1$, use 512 repetitions, and vary $\epsilon \in \{0, 0.1, 0.2, 0.5, 1.0\}$, with $\epsilon = 0$ being the configuration providing the exact minimum spanning tree with probability $\geq .9$.

Table 1 reports the running time, in seconds, of all algorithms executed on all datasets, while Table 2 reports the observed percentage relative error $100(c(T)/c(T^*) - 1)\%$, where T is the returned tree and T^* is an EMST. Runs marked by a - time out after 24 hours. First, considering the exact setting, we note that our algorithm is able to find the exact EMST 37–56× faster than *TUTTE* on the four datasets where it completes, which in turn is much faster than *MLPACK*. In particular, on million-scale datasets *wok-stir* finds the EMST in ≈ 10 minutes (*sift*) and about 36 minutes (*glove*), where *TUTTE* either requires over 8 hours or times out. We note that the running times of *MLPACK* reflect its single-threaded implementation. However, even assuming perfect scaling of the running times (thus dividing the runtime by the 32 cores), *MLPACK* would still be slower than *wok-stir*. This shows that our algorithmic improvements are

³<https://github.com/Cecca/panna>, the Playground for Approximate Nearest Neighbors Algorithms.

⁴https://github.com/TutteInstitute/fast_hdbscan/

⁵<https://github.com/lmcinnes/pynndescent>

Table 1: Running times in seconds for finding the Euclidean Minimum Spanning Tree for different algorithms. For WOK-STIR we report the time under different values of the approximation parameter ε . Note that $\varepsilon = 0$ is our *exact* algorithm. In parentheses we report the percentage of distance computations, with respect to the $\binom{n}{2}$ possible pairs of points.

	TUTTE		MLPACK	HSSL	WOK-STIR									
dataset	exact	approx	exact	approx	exact $\varepsilon = 0$		$\varepsilon = 0.1$		$\varepsilon = 0.2$		$\varepsilon = 0.5$		$\varepsilon = 1$	
fashion	730.7	22.1	5175.1	16.5	19.6	(19.6%)	5.2	(7.9%)	3.8	(6.1%)	3.1	(3.7%)	2.6	(1.9%)
mnist	811.6	26.5	8723.9	20.7	15.4	(28.6%)	6.1	(4.5%)	4.6	(2.6%)	2.8	(0.9%)	2.8	(0.7%)
nytimes	9102.8	48.4	-	101.9	223.5	(374.3%)	100.5	(179.2%)	49.9	(71.8%)	15.4	(5.4%)	8.4	(0.2%)
sift	30934.5	92.4	-	212.3	550.3	(16.3%)	109.0	(2.6%)	61.2	(1.1%)	31.6	(0.3%)	25.2	(0.2%)
glove	-	131.3	-	276.5	2161.2	(305.0%)	652.4	(110.6%)	234.4	(23.4%)	100.4	(3.4%)	36.9	(0.0%)

Table 2: Percentage relative error on the weight of the found tree, with respect to the weight of the EMST.

dataset	TUTTE	HSSL	WOK-STIR			
	approx		$\varepsilon = 0.1$	$\varepsilon = 0.2$	$\varepsilon = 0.5$	$\varepsilon = 1$
fashion	0.02%	0.28%	0.24%	0.47%	1.02%	2.18%
mnist	0.01%	0.06%	0.12%	0.46%	2.99%	6.46%
nytimes	3.83%	2.65%	0.00%	0.02%	1.14%	7.68%
sift	0.27%	0.24%	0.04%	0.23%	1.54%	3.21%
glove	1.75%	0.94%	0.00%	0.03%	0.75%	7.27%

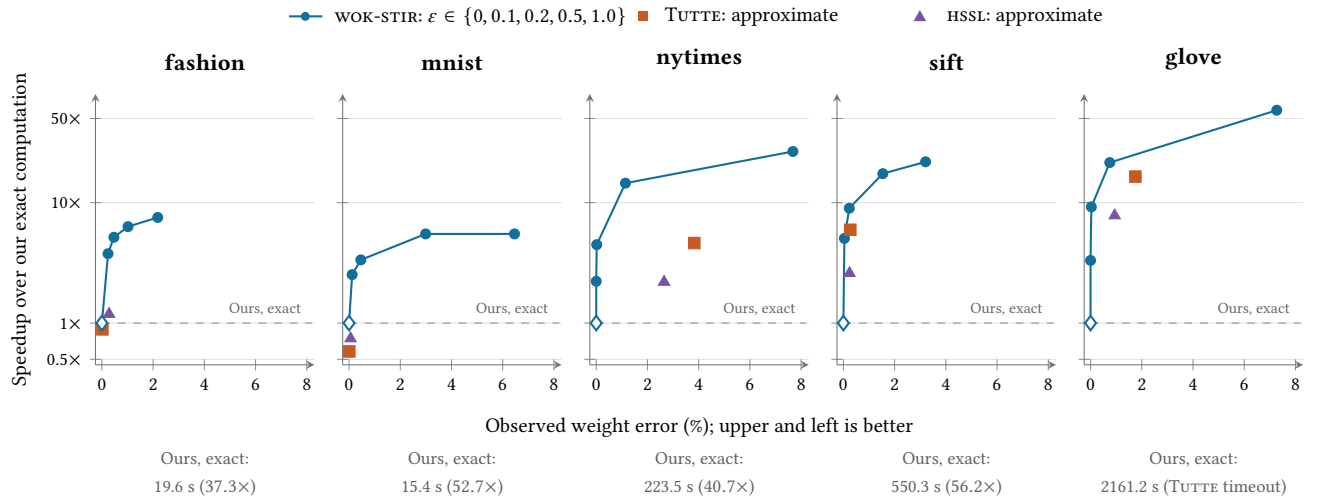


Figure 7: EMST runtime–quality tradeoff from Tables 1 and 2. The vertical axis shows speedup over our exact computation; the horizontal axis shows observed relative weight error. Connected points represent $\varepsilon \in \{0, 0.1, 0.2, 0.5, 1\}$; hollow diamonds and the dashed 1 $\times$ line mark our exact setting. Beneath each panel, we report our exact runtime and, in parentheses, its speedup over TUTTE, the fastest completed exact baseline.

indeed effective in allowing efficient computation of the EMST in the exact setting.

In the approximate setting, WOK-STIR allows us to exercise a *tradeoff* between running time and quality, visualized in Figure 7. The vertical axis shows speedup over our exact computation, while the horizontal axis shows observed relative error.

We note that by design the parameter ε provides a *guaranteed* upper bound on the relative error: $\varepsilon = 0.5$ guarantees a relative error $\leq 50\%$: empirically, the actual relative error is much smaller.

Furthermore, WOK-STIR allows us to further reduce the running time by being more permissive with the error. In particular, on million-scale datasets (sift and glove), WOK-STIR finds spanning trees with 3.21% and 7.27% relative error in about 25 and 37 seconds, respectively.

Among the approximate baselines, TUTTE shows impressive gains in performance, with a modest (empirical) loss of quality in terms of relative error. HSSL also achieves low error on these datasets, and comparable running time on the two smallest ones.

Table 3: Percentage relative error on the synthetic dataset generated with DENSIRED.

	percentage relative error
TUTTE (exact)	0.00%
MLPACK (exact)	0.00%
WOK-STIR ($\epsilon=0.0$, exact)	0.00%
WOK-STIR ($\epsilon=0.1$)	0.52%
WOK-STIR ($\epsilon=0.2$)	3.36%
WOK-STIR ($\epsilon=0.5$)	21.79%
WOK-STIR ($\epsilon=1.0$)	32.42%
HSSL (approx)	39.68%
TUTTE (approx)	445.07%

On larger datasets, TUTTE scales better than HSSL. For our algorithm, on smaller datasets (fashion and mnist), the exact setting of WOK-STIR is comparable to or faster than the *approximate* configuration of TUTTE; on mnist, it is also faster than HSSL. On larger datasets, WOK-STIR can achieve comparable quality and running time compared to the baselines: in many configurations it is both *faster* and *more accurate* than the baselines.

Table 1 reports, in parentheses, the percentage of distance computations over $\binom{n}{2}$ carried out by our algorithm. Notably, on nytimes and glove in the exact setting the algorithm performs *more* distance computations than $\binom{n}{2}$, due to duplicate collisions. Increasing ϵ significantly reduces the number of distance computations on every dataset, as expected from the theoretical analysis. In the most extreme case (glove with $\epsilon = 1$) the distances computed are 0.05% of the possible pairs of points (i.e. ≈ 347 million out of ≈ 700 billion pairs).

To investigate the robustness of different algorithms with respect to quality, we generated a synthetic dataset using DENSIRED [25]. In particular, we generate 5000 tight, well-separated clusters (generator’s parameters `core_num=1`, `radius=0.15`) over 100 000 points in 100 dimensions. In this dataset the EMST contains edges that *do not* connect near neighbors.

Table 3 shows that WOK-STIR, increasing the parameter ϵ , finds trees whose cost is progressively larger than the minimum, in a controlled way. HSSL finds a tree with $\approx 40\%$ relative error. Interestingly, TUTTE in the approximate configuration finds a tree with approximately 445% relative error, i.e., whose cost is over 5 times the minimum. The reason is that it builds a tree out of an approximate k -nearest neighbor graph that does not contain the inter-cluster edges that are necessary to build the tree. It then falls back to adding arbitrary edges to the tree, resulting in the high relative error observed in this experiment⁶.

7.3 Mutual Reachability Distance (RQ3)

Here we consider the computation of the EMST under the mutual reachability distance, under different values `minPts` $\in \{5, 15, 30, 60, 120\}$ of the parameter defining the core distance.

⁶In the context of clustering this might be irrelevant: clusters are well separated and using arbitrary edges to connect them in the dendrogram does little damage. For finding the EMST, though, this is substantial.

Table 4: Running times in seconds for finding the EMST under the mutual-reachability distance, under different minPts values. Our algorithm is executed under different ϵ settings. For WOK-STIR, parentheses show speedups over the faster baseline in each row, $\min(t_{\text{TUTTE}}, t_{\text{HSSL}})/t_{\text{WOK-STIR}}$. Values below $1\times$ indicate a slowdown.

	minPts	TUTTE	HSSL	WOK-STIR: time (speedup)		
		approx	approx	$\epsilon = 0.1$	$\epsilon = 0.5$	$\epsilon = 1.0$
fashion	5	34.7	28.8	8.3 (3.5 $\times$)	4.8 (6.0 $\times$)	3.8 (7.6 $\times$)
	15	34.7	68.3	11.3 (3.1 $\times$)	5.1 (6.8 $\times$)	4.1 (8.4 $\times$)
	30	51.2	137.4	14.1 (3.6 $\times$)	6.0 (8.6 $\times$)	4.8 (10.6 $\times$)
	60	67.4	261.3	17.1 (3.9 $\times$)	7.0 (9.7 $\times$)	5.0 (13.5 $\times$)
	120	99.3	523.5	25.1 (4.0 $\times$)	10.5 (9.5 $\times$)	7.3 (13.6 $\times$)
mnist	5	26.9	32.3	9.4 (2.9 $\times$)	4.3 (6.2 $\times$)	3.4 (7.9 $\times$)
	15	33.3	81.7	14.6 (2.3 $\times$)	5.2 (6.4 $\times$)	4.2 (7.9 $\times$)
	30	48.7	178.9	20.1 (2.4 $\times$)	6.3 (7.7 $\times$)	4.5 (10.9 $\times$)
	60	76.2	400.4	27.4 (2.8 $\times$)	9.1 (8.3 $\times$)	6.2 (12.2 $\times$)
	120	108.0	745.0	41.2 (2.6 $\times$)	17.7 (6.1 $\times$)	10.8 (10.0 $\times$)
nytimes	5	49.1	125.2	127.6 (0.4 $\times$)	24.3 (2.0 $\times$)	12.5 (3.9 $\times$)
	15	75.7	300.4	171.0 (0.4 $\times$)	32.2 (2.3 $\times$)	16.9 (4.5 $\times$)
	30	127.7	636.9	220.4 (0.6 $\times$)	43.7 (2.9 $\times$)	24.9 (5.1 $\times$)
	60	271.9	1,393.2	271.4 (1.0 $\times$)	73.5 (3.7 $\times$)	49.9 (5.5 $\times$)
	120	583.8	3,027.3	368.9 (1.6 $\times$)	176.6 (3.3 $\times$)	133.0 (4.4 $\times$)
sift	5	98.8	382.8	341.8 (0.3 $\times$)	55.4 (1.8 $\times$)	36.0 (2.7 $\times$)
	15	179.2	1,050.5	596.1 (0.3 $\times$)	76.9 (2.3 $\times$)	41.9 (4.3 $\times$)
	30	330.0	2,192.1	739.7 (0.4 $\times$)	116.9 (2.8 $\times$)	61.2 (5.4 $\times$)
	60	706.8	4,434.9	1,095.8 (0.6 $\times$)	185.7 (3.8 $\times$)	98.8 (7.2 $\times$)
	120	1,347.9	20,675.5	1,441.8 (0.9 $\times$)	313.5 (4.3 $\times$)	159.8 (8.4 $\times$)
glove	5	140.4	516.0	1,391.9 (0.1 $\times$)	209.8 (0.7 $\times$)	67.2 (2.1 $\times$)
	15	247.9	1,301.3	2,532.5 (0.1 $\times$)	325.0 (0.8 $\times$)	98.2 (2.5 $\times$)
	30	441.5	3,055.2	3,066.0 (0.1 $\times$)	493.4 (0.9 $\times$)	148.2 (3.0 $\times$)
	60	948.9	5,646.2	3,738.6 (0.3 $\times$)	803.9 (1.2 $\times$)	255.7 (3.7 $\times$)
	120	2,352.2	29,800.9	5,872.4 (0.4 $\times$)	1,191.0 (2.0 $\times$)	429.9 (5.5 $\times$)

Since it does not natively support mutual reachability distances, we exclude MLPACK from this comparison. Among the other two baselines, TUTTE consistently outperforms HSSL, as reported in Table 4. As for WOK-STIR in the approximate setting, on smaller datasets (mnist and fashion) it is always faster than the baselines. Using a larger value for ϵ gives faster running times on all datasets at only a moderate decrease in quality, as we shall see in the next section. On larger datasets, we note that WOK-STIR scales better with the *minPts* parameter compared to the baselines, being up to ≈ 5.4 times faster.

7.4 HDBSCAN Dendrogram Comparison (RQ4)

To compare the quality of the clustering induced by the trees computed by different methods, we compare the resulting dendrograms, rather than any flat clustering that could be extracted from the dendrogram, which in fact depends on the flattening algorithm. To compare dendrograms we use the *cophenetic* distance [36]: for two points x, y , it is the weight of the heaviest edge in the simple path in the tree connecting x and y . A large cophenetic distance implies that two points are merged in the same cluster close to the root

Table 5: Spearman correlation coefficient ($\uparrow$) of the cophenetic distance between data points, with respect to the dendrogram built by WOK-STIR with $\varepsilon = 0$. Numbers in parentheses report the mean absolute relative error of the cophenetic distance.

	minPts	TUTTE	HSSL	WOK-STIR		
		approx	approx	0.1	0.5	1.0
fashion	5	1.0 (0.0%)	0.92 (3.4%)	0.99 (0.2%)	0.98 (1.7%)	0.97 (3.0%)
	15	1.0 (0.0%)	0.94 (4.3%)	0.99 (0.3%)	0.95 (2.7%)	0.93 (5.5%)
	30	1.0 (0.0%)	0.94 (4.9%)	0.98 (0.3%)	0.93 (3.5%)	0.91 (6.5%)
	60	1.0 (0.0%)	0.96 (4.9%)	0.98 (0.3%)	0.92 (4.8%)	0.9 (9.7%)
	120	1.0 (0.0%)	0.96 (5.3%)	0.98 (0.4%)	0.9 (6.0%)	0.89 (12.1%)
mnist	5	1.0 (0.0%)	0.91 (4.1%)	1.0 (0.0%)	0.93 (4.7%)	0.89 (12.5%)
	15	1.0 (0.0%)	0.97 (4.7%)	1.0 (0.0%)	0.97 (3.5%)	0.9 (14.5%)
	30	1.0 (0.0%)	0.98 (4.6%)	1.0 (0.0%)	0.97 (3.6%)	0.89 (17.7%)
	60	1.0 (0.0%)	0.98 (4.5%)	1.0 (0.0%)	0.97 (3.2%)	0.89 (19.9%)
	120	1.0 (0.0%)	0.98 (4.5%)	1.0 (0.0%)	0.97 (3.1%)	0.9 (17.8%)
nytimes	5	0.9 (2.6%)	0.68 (8.2%)	1.0 (0.0%)	0.99 (0.9%)	0.84 (7.0%)
	15	1.0 (0.4%)	0.74 (7.5%)	1.0 (0.0%)	0.99 (0.7%)	0.85 (5.9%)
	30	1.0 (0.2%)	0.76 (6.3%)	1.0 (0.0%)	1.0 (0.5%)	0.9 (3.9%)
	60	1.0 (0.1%)	0.72 (6.5%)	1.0 (0.0%)	1.0 (0.4%)	0.93 (2.9%)
	120	1.0 (0.1%)	0.62 (6.6%)	1.0 (0.0%)	1.0 (0.1%)	1.0 (0.4%)
sift	5	1.0 (0.3%)	0.84 (5.8%)	1.0 (0.0%)	0.98 (2.4%)	0.96 (4.8%)
	15	1.0 (0.0%)	0.87 (7.3%)	1.0 (0.0%)	0.98 (2.3%)	0.96 (6.5%)
	30	1.0 (0.0%)	0.82 (7.5%)	1.0 (0.0%)	0.98 (2.5%)	0.95 (8.1%)
	60	1.0 (0.0%)	0.77 (7.6%)	1.0 (0.0%)	0.99 (2.1%)	0.91 (8.7%)
	120	1.0 (0.0%)	0.99 (1.2%)	1.0 (0.0%)	0.98 (1.4%)	0.87 (8.8%)
glove	5	0.97 (2.4%)	0.81 (9.4%)	1.0 (0.0%)	0.99 (0.7%)	0.95 (5.5%)
	15	1.0 (0.5%)	0.89 (9.5%)	1.0 (0.0%)	1.0 (0.6%)	0.97 (4.9%)
	30	1.0 (0.3%)	0.91 (7.8%)	1.0 (0.0%)	1.0 (0.5%)	0.97 (4.9%)
	60	1.0 (0.2%)	0.92 (6.8%)	1.0 (0.0%)	1.0 (0.4%)	0.97 (4.7%)
	120	1.0 (0.1%)	0.98 (1.3%)	1.0 (0.0%)	1.0 (0.4%)	0.96 (4.6%)

of the dendrogram. As such, if two methods produce trees with a similar cophenetic distance for any pair of points, then the structure of the hierarchical clustering is similar, which can be captured by the Spearman correlation coefficient of the cophenetic distances induced by two trees: a high correlation indicates that the overall structure of the dendrograms aligns well.

Table 5 reports the Spearman correlation coefficient of the cophenetic distances induced by trees built by TUTTE, HSSL and WOK-STIR with respect to the exact ground truth dendrograms. In parentheses the table reports the mean absolute relative error (MARE) of the cophenetic distances. In general, the dendrograms computed by TUTTE and WOK-STIR are quite similar to the reference one, as indicated by the high correlation, with TUTTE and WOK-STIR with $\varepsilon = 0.1$ producing effectively the same clustering as the reference.

8 Conclusion

We presented an LSH-based framework for exact and approximate Euclidean minimum spanning tree computation with probabilistic guarantees, and extended it to mutual reachability distances used for HDBSCAN. Our adaptive stopping rules certify solution quality, while our analysis relates computational work to the input’s

distance distribution and proposes novel difficulty measures. Experiments on high-dimensional datasets demonstrate substantial speedups and a practical tradeoff between computation time and tree quality. An evaluation on synthetic workloads further illustrates the value of explicit approximation guarantees since heuristic methods may return trees with large errors.

Acknowledgments

This work was supported in part by MUR PNRR CN00000013 National Center for HPC, Big Data and Quantum Computing.

References

- [1] Alexandr Andoni and Piotr Indyk. 2006. Near-Optimal Hashing Algorithms for Approximate Nearest Neighbor in High Dimensions. In *FOCS*. IEEE Computer Society, 459–468.
- [2] Alexandr Andoni, Piotr Indyk, Thijs Laarhoven, Ilya P. Razenshteyn, and Ludwig Schmidt. 2015. Practical and Optimal LSH for Angular Distance. In *NIPS*. 1225–1233.
- [3] Sunil Arya and David M Mount. 2016. A fast and simple algorithm for computing approximate Euclidean minimum spanning trees. In *Proceedings of the Twenty-Seventh Annual ACM-SIAM Symposium on Discrete Algorithms*. SIAM, 1220–1233.
- [4] Martin Aumüller and Matteo Ceccarello. 2022. Implementing Distributed Similarity Joins using Locality Sensitive Hashing. In *EDBT*. OpenProceedings.org, 1:78–1:90.
- [5] Martin Aumüller and Matteo Ceccarello. 2023. Solving k-Closest Pairs in High-Dimensional Data. In *SISAP (Lecture Notes in Computer Science)*, Vol. 14289. Springer, 200–214.
- [6] Martin Aumüller, Tobias Christiani, Rasmus Pagh, and Michael Vesterli. 2019. PUFFINN: Parameterless and Universally Fast Finding of Nearest Neighbors. In *ESA (LIPIcs)*, Vol. 144. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 10:1–10:16.
- [7] Martin Aumüller, Erik Bernhardsson, and Alexander Faithfull. 2018. ANN-Benchmarks: A Benchmarking Tool for Approximate Nearest Neighbor Algorithms. arXiv:1807.05614 [cs.LG] <https://arxiv.org/abs/1807.05614>
- [8] Mayank Bawa, Tyson Condie, and Prasanna Ganesan. 2005. LSH forest: self-tuning indexes for similarity search. In *WWW*. ACM, 651–660.
- [9] Ricardo JGB Campello, Davoud Moulavi, and Jörg Sander. 2013. Density-based clustering based on hierarchical density estimates. In *Pacific-Asia conference on knowledge discovery and data mining*. Springer, 160–172.
- [10] Matteo Ceccarello, Francesco Pio Monaco, and Francesco Silvestri. 2025. MO-MENTI: Scalable Motif Mining in Multidimensional Time Series. *Proc. VLDB Endow.* 18, 10 (2025), 3463–3476. <https://doi.org/10.14778/3748191.3748208>
- [11] Samidh Chatterjee, Michael Connor, and Piyush Kumar. 2009. *Computing Geometric Minimum Spanning Trees Using the Filter-Kruskal Method*. Technical Report. Citeseer.
- [12] Flavio Chierichetti and Ravi Kumar. 2015. LSH-Preserving Functions and Their Applications. *J. ACM* 62, 5 (2015), 33:1–33:25.
- [13] Sun Chung and A. Condon. 1996. Parallel implementation of Bouvka’s minimum spanning tree algorithm. In *Proceedings of International Conference on Parallel Processing*. 302–308. <https://doi.org/10.1109/IPPS.1996.508073>
- [14] John Conway and Neil Sloane. 1982. Fast quantizing and decoding and algorithms for lattice quantizers and codes. *IEEE Transactions on Information Theory* 28, 2 (1982), 227–232.
- [15] Ryan R. Curtin, Marcus Edel, Omar Shrit, Shubham Agrawal, Suryoday Basak, James J. Balamuta, Ryan Birmingham, Kartik Dutt, Dirk Eddelbuettel, Rishabh Garg, Shikhar Jaiswal, Aakash Kaushik, Sangyeon Kim, Anjishnu Mukherjee, Nanubala Gnana Sai, Nippun Sharma, Yashwant Singh Parihar, Roshan Swain, and Conrad Sanderson. 2023. mlpack 4: a fast, header-only C++ machine learning library. *Journal of Open Source Software* 8, 82 (2023), 5026. <https://doi.org/10.21105/joss.05026>
- [16] Artur Czumaj, Funda Ergün, Lance Fortnow, Avner Magen, Ilan Newman, Ronitt Rubinfeld, and Christian Sohler. 2005. Approximating the weight of the euclidean minimum spanning tree in sublinear time. *SIAM J. Comput.* 35, 1 (2005), 91–109.
- [17] Mayur Datar, Nicole Immorlica, Piotr Indyk, and Vahab S. Mirrokni. 2004. Locality-sensitive hashing scheme based on p-stable distributions. In *SCG*. ACM, 253–262.
- [18] Mark de Berg, Ade Gunawan, and Marcel Roeloffzen. 2019. Faster dbscan and hdbscan in low-dimensional euclidean spaces. *International Journal of Computational Geometry & Applications* 29, 01 (2019), 21–47.
- [19] Martin Ester, Hans-Peter Kriegel, Jörg Sander, and Xiaowei Xu. 1996. A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noise. In *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining (KDD)*. AAAI Press, 226–231.

- [20] Teofilo F. Gonzalez. 1985. Clustering to Minimize the Maximum Intercluster Distance. *Theor. Comput. Sci.* 38 (1985), 293–306.
- [21] J. C. Gower and G. J. S. Ross. 1969. Minimum Spanning Trees and Single Linkage Cluster Analysis. *Journal of the Royal Statistical Society. Series C (Applied Statistics)* 18, 1 (1969), 54–64. <https://doi.org/10.2307/2346439> Publisher: [Wiley, Royal Statistical Society].
- [22] Sarel Har-Peled, Piotr Indyk, and Rajeev Motwani. 2012. Approximate nearest neighbor: Towards removing the curse of dimensionality. (2012).
- [23] Piotr Indyk. 2004. Algorithms for dynamic geometric problems over data streams. In *Proceedings of the thirty-sixth annual ACM Symposium on Theory of Computing*. 373–380.
- [24] Piotr Indyk and Rajeev Motwani. 1998. Approximate Nearest Neighbors: Towards Removing the Curse of Dimensionality. In *STOC*. ACM, 604–613.
- [25] Philipp Jahn, Christian M. M. Frey, Anna Beer, Collin Leiber, and Thomas Seidl. 2024. Data with Density-Based Clusters: A Generator for Systematic Evaluation of Clustering Algorithms. In *ECML/PKDD (7) (Lecture Notes in Computer Science)*, Vol. 14947. Springer, 3–21.
- [26] R Jothi, Sraban Kumar Mohanty, and Aparajita Ojha. 2016. Functional grouping of similar genes using eigenanalysis on minimum spanning tree based neighborhood graph. *Computers in biology and medicine* 71 (2016), 135–148.
- [27] Yury A. Malkov and Dmitry A. Yashunin. 2020. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Trans. Pattern Anal. Mach. Intell.* 42, 4 (2020), 824–836.
- [28] William B March, Parikshit Ram, and Alexander G Gray. 2010. Fast euclidean minimum spanning tree: algorithm, analysis, and applications. In *Proceedings of the 16th ACM SIGKDD international conference on Knowledge discovery and data mining*. 603–612.
- [29] Artur Mariano, Alberto Proenca, and Cristiano Da Silva Sousa. 2015. A Generic and Highly Efficient Parallel Variant of Boruvka’s Algorithm. In *2015 23rd Euromicro International Conference on Parallel, Distributed, and Network-Based Processing*. 610–617. <https://doi.org/10.1109/PDP.2015.72>
- [30] Leland McInnes and John Healy. 2017. Accelerated Hierarchical Density Based Clustering. In *2017 IEEE International Conference on Data Mining Workshops (ICDMW)*. IEEE, 33–42. <https://doi.org/10.1109/ICDMW.2017.12>
- [31] Camilla Birch Okkels, Erik Thordsen, Martin Aumüller, Arthur Zimek, and Erich Schubert. 2025. Approximate single-linkage clustering using graph-based indexes: MST-based approaches and incremental searchers. In *International Conference on Similarity Search and Applications*. Springer, 233–247.
- [32] Camilla Birch Okkels, Erik Thordsen, Martin Aumüller, Arthur Zimek, and Erich Schubert. 2026. Approximate hierarchical density-based clustering using graph-based search indexes. *Inf. Syst.* 142 (2026), 102768.
- [33] Tim Roughgarden (Ed.). 2020. *Beyond the Worst-Case Analysis of Algorithms*. Cambridge University Press.
- [34] Michael Ian Shamos and Dan Hoey. 1975. Closest-point problems. In *16th Annual Symposium on Foundations of Computer Science (sfcs 1975)*. 151–162. <https://doi.org/10.1109/SFCS.1975.8>
- [35] Daniel Dominic Sleator and Robert Endre Tarjan. 1983. A Data Structure for Dynamic Trees. *J. Comput. Syst. Sci.* 26, 3 (1983), 362–391.
- [36] Robert R. Sokal and F. James Rohlf. 1962. The Comparison of Dendrograms by Objective Methods. *Taxon* 11, 2 (1962), 33–40. <https://doi.org/10.2307/1217208> Publisher: International Association for Plant Taxonomy (IAPT).
- [37] Pravin M Vaidya. 1991. A sparse graph almost as good as the complete graph on points in K dimensions. *Discrete & Computational Geometry* 6, 3 (1991), 369–381.
- [38] P. J. Wan, G. Călinescu, X. Y. Li, and O. Frieder. 2002. Minimum-energy broadcasting in static ad hoc wireless networks. *Wirel. Netw.* 8, 6 (Nov. 2002), 607–617. <https://doi.org/10.1023/A:1020381720601>
- [39] Yiqiu Wang, Shangdi Yu, Yan Gu, and Julian Shun. 2021. Fast parallel algorithms for euclidean minimum spanning tree and hierarchical spatial clustering. In *Proceedings of the 2021 international conference on management of data*. 1982–1995.